

Clase 013 — HTTP, HTTPS y la arquitectura de la web moderna

Parte: **0 — Fundamentos y prerequisites** · Fuente: *RFC 9110 (HTTP Semantics) / MDN Web Docs* 
Duración estimada: **110 min** · Nivel: **Fundamentos**

Objetivo

Dominar el protocolo sobre el que corre la web y, con él, la mayor parte de la superficie de ataque moderna. Al terminar entenderás peticiones y respuestas HTTP, métodos, códigos de estado, cabeceras, cookies y sesiones, y cómo HTTPS/TLS añade confidencialidad e integridad al canal.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Construir** e interpretar peticiones y respuestas HTTP.
2. **Explicar** métodos, códigos de estado y cabeceras relevantes.
3. **Describir** cómo funcionan cookies, sesiones y autenticación web.
4. **Explicar** qué aporta TLS y cómo se establece una conexión HTTPS.
5. **Analizar** tráfico web con herramientas de interceptación.

Temas

#	Tema	Por qué importa
1	Modelo petición/respuesta	Base de toda interacción web
2	Métodos HTTP	GET, POST, PUT, DELETE y su semántica
3	Códigos de estado	2xx/3xx/4xx/5xx diagnostican todo
4	Cabeceras	Control, seguridad y contexto
5	Cookies y sesiones	Estado sobre un protocolo sin estado
6	HTTPS/TLS	Confidencialidad e integridad
7	Cabeceras de seguridad	HSTS, CSP, etc.
8	HTTP/2 y HTTP/3	La web moderna

Definiciones y características

- **HTTP**: protocolo de aplicación sin estado sobre TCP (o QUIC en HTTP/3). Clave: cada petición es independiente; el estado se añade con cookies.

- **Método:** verbo que indica la acción (GET lee, POST envía, PUT reemplaza, DELETE borra). Clave: GET debe ser seguro/idempotente.
- **Código de estado:** número de la respuesta (200 OK, 301 redirección, 401/403 auth, 404, 500). Clave: primer diagnóstico de cualquier fallo web.
- **Cookie:** dato que el servidor pide al navegador guardar y reenviar. Clave: `HttpOnly`, `Secure`, `SameSite` la protegen.
- **TLS:** protocolo que cifra y autentica el canal mediante certificados. Clave: HTTPS = HTTP sobre TLS; protege confidencialidad e integridad, no la app.
- **HSTS:** cabecera que fuerza HTTPS en el navegador. Clave: mitiga el *downgrade* a HTTP.

Herramientas y preparación

Usa `curl`, el navegador con sus **DevTools** (pestaña Network), y un proxy de interceptación como **Burp Suite Community** o **OWASP ZAP**. Para inspeccionar TLS, `openssl s_client`. Practica contra una app de laboratorio (DVWA, o un servidor propio), nunca contra sitios ajenos sin permiso.

Laboratorio guiado

1. **Petición cruda con curl.** Observa cabeceras de respuesta:

```
bash curl -v http://10.10.10.6/ 2>&1 | head -40
```

Identifica la línea de estado, cabeceras y cuerpo. 2. **Métodos y estados.** Prueba distintos métodos y observa el código:

```
bash curl -s -o /dev/null -w "%{http_code}\n" -X POST http://10.10.10.6/login
```

3. **Inspeccionar cookies.** Haz login en la app de laboratorio con DevTools abiertas y revisa la cookie de sesión: ¿tiene `HttpOnly`, `Secure`, `SameSite`?
4. **Interceptar con Burp/ZAP.** Configura el navegador para pasar por el proxy y captura una petición; modifícala y reenvíala (Repeater) para ver el efecto en el servidor.
5. **Examinar TLS:**

```
bash openssl s_client -connect example.com:443 -servername example.com </dev/null 2>/dev/null | openssl x509 -noout -subject -issuer -dates
```

Lee el emisor y la validez del certificado. 6. **Cabeceras de seguridad.** Comprueba si un sitio envía HSTS/CSP:

```
bash curl -sI https://example.com | grep -iE 'strict-transport|content-security'
```

 **Nota ética:** la interceptación y manipulación de tráfico web se realiza **solo** contra aplicaciones propias o autorizadas. Usar un proxy contra sitios de terceros sin permiso es ilegal.

Ejercicios

1. Clasifica estos códigos: 204, 301, 401, 403, 429, 502. ¿Qué significa cada uno?
2. Explica la diferencia entre 401 y 403.
3. ¿Qué atributos hacen segura una cookie de sesión y contra qué protege cada uno?

4. Describe qué información revela y qué oculta TLS a un observador de la red.
5. Investiga la cabecera `Content-Security-Policy` y da un ejemplo que mitigue XSS.
6. Con Burp Repeater, cambia un parámetro y documenta cómo responde la app.

Reto verificable

Usando un proxy de interceptación, captura el flujo completo de autenticación de una app de laboratorio: la petición de login, la respuesta que establece la cookie de sesión, y una petición autenticada posterior. Documenta las cabeceras de seguridad presentes y ausentes, y propón mejoras.

Criterio de aceptación: la evidencia muestra la cookie de sesión con sus atributos reales, e identificas correctamente al menos dos cabeceras de seguridad faltantes (p. ej. HSTS, CSP, `SameSite`) con una recomendación concreta por cada una. Reproducible contra la misma app.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Burp no intercepta HTTPS	Falta instalar el certificado CA de Burp en el navegador. Impórtalo.
<code>curl</code> da error de certificado	Certificado autofirmado en el lab. Usa <code>-k</code> solo en laboratorio, nunca en producción.
La cookie no se envía en peticiones	Atributo <code>SameSite</code> / <code>Secure</code> o dominio/ruta incorrectos. Revisa el scope.
405 Method Not Allowed	El recurso no admite ese método. Comprueba la API.
HTTP/2 no aparece en la captura	Negociado por ALPN dentro de TLS. Usa herramientas que lo soporten.

Preguntas frecuentes

 **¿HTTPS hace segura mi aplicación?** No: cifra el canal, pero no protege contra fallos de la app (inyección, XSS, lógica). Es necesario pero no suficiente.

 **¿Por qué HTTP es "sin estado" si hay sesiones?** El protocolo no recuerda peticiones anteriores; las cookies y tokens simulan estado guardándolo en cliente/servidor.

 **¿Qué cambia HTTP/2 y HTTP/3?** HTTP/2 multiplexa varias peticiones en una conexión; HTTP/3 corre sobre QUIC (UDP) reduciendo latencia. La semántica (métodos, estados) se mantiene.

 **¿Burp o ZAP?** Ambos interceptan y modifican tráfico. Burp es el estándar profesional; ZAP es open source y gratuito. Para aprender, cualquiera sirve.

Referencias

- RFC 9110, *HTTP Semantics* — <https://www.rfc-editor.org/rfc/rfc9110>
- MDN Web Docs: HTTP — <https://developer.mozilla.org/docs/Web/HTTP>
- OWASP Secure Headers Project — <https://owasp.org/www-project-secure-headers/>
- PortSwigger Web Security Academy — <https://portswigger.net/web-security>

Siguiente clase

Clase 014 - Direccionamiento IP y subnetting